

LAPORAN PROJEK

IMPLEMENTASI DAN ANALISIS KOMPARATIF ALGORITMA QUICK SORT DAN EXPONENTIAL SEARCH PADA SISTEM TRANSAKSI E-COMMERCE

Disusun guna memenuhi tugas mata kuliah struktur data

Dosen pengampu:

Ali Akbar Lubis, S.Kom., M.TI.



Disusun Oleh:

KELOMPOK 1 PTIK C 2025:

CHESSA RUTH OLIVIA PANGARIBUAN	(5253151023)
DANIEL VIDIARGA MANIK	(5253151015)
FADHIL SATRIA ZEBUA	(5253151005)

PROGRAM STUDI PENDIDIKAN TEKNOLOGI INFORMATIKA DAN KOMPUTER

FAKULTAS TEKNIK

UNIVERSITAS NEGERI MEDAN

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat-Nya laporan tugas proyek Struktur Data berjudul "Implementasi dan Analisis Komparatif Algoritma Quick Sort dan Exponential Search pada Sistem Transaksi E-Commerce" ini dapat selesai tepat waktu. Laporan ini disusun untuk memenuhi tugas kelompok wajib pada mata kuliah Struktur Data dengan melakukan implementasi mandiri (from scratch) serta pengujian empiris terhadap efisiensi algoritma Quick Sort dan Exponential Search menggunakan dataset transaksi ritel nyata. Penulis menyampaikan terima kasih yang sebesar-besarnya kepada Bapak Ali Akbar Lubis, S.Kom., M.TI. selaku dosen pengampu atas segala arahan teknisnya, serta kepada seluruh anggota Kelompok 1 PTIK C 2025 dan rekan mahasiswa Universitas Negeri Medan atas kerja sama dan dukungan akademis yang diberikan. Penulis menyadari bahwa laporan ini masih memiliki keterbatasan, sehingga kritik dan saran yang membangun sangat diharapkan demi penyempurnaan di masa mendatang. Semoga laporan proyek ini dapat memberikan manfaat akademik yang berguna bagi para pembaca.

Medan, 3 Juni 2026

Kelompok 1

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I	1
PENDAHULUAN.....	1
1.1 Latar belakang	1
1.2 Rumusan masalah	1
1.3 Tujuan proyek	2
BAB II	3
LANDASAN TEORI	3
2.1 Algoritma bubble sort (sebagai pembandingan).....	3
2.2 Algoritma quick sort	3
2.3 Algoritma exponential search	4
2.4 Analisis kompleksitas teoretis (Big-O).....	5
2.5 Kelebihan dan Kekurangan Algoritma	5
BAB III.....	7
PSEUDOCODE.....	7
3.1 Pseudocode modul quick sort	7
3.2 Pseudocode modul bubble sort	8
BAB IV.....	10
DESAIN DAN IMPLEMENTASI PROGRAM	10
4.1 Arsitektur program modular	10

4.2 Struktur data objek transaksi (struct).....	10
4.3 Penanganan Kasus Ekstrem dan Keamanan Memori (Robustness)	11
4.4 Desain Antarmuka Pengguna (CLI Layout)	11
BAB V	13
PANDUAN STUDI KASUS	13
5.1 Deskripsi Kasus Nyata Industri E-Commerce	13
5.2 Karakteristik Sampel Dataset Ritel.....	13
5.4 Skenario Kasus Pengujian Fungsional.....	14
BAB VI.....	15
ANALISIS HASIL EKSPERIMEN	15
6.1 Pengujian Performa Waktu Eksekusi (Benchmark).....	15
6.2 Pengujian Jumlah Operasi Berdasarkan Kondisi Variasi Input Data.....	19
6.3 Pembahasan Temuan Ilmiah dan Trade-off Performa	21
6.4 Validasi Hasil dan Batasan Eksperimen (Sub-bab Baru).....	23
6.5 Solusi Optimalisasi Masalah Kasus Terburuk (Worst-case).....	24
BAB VII	25
KESIMPULAN DAN REKOMENDASI	25
DAFTAR PUSTAKA	28

BAB I

PENDAHULUAN

1.1 Latar belakang

Dalam era digitalisasi komparatif modern, aktivitas industri e-commerce menghasilkan akumulasi data transaksi dalam volume yang sangat masif setiap harinya. Pengelolaan data berskala besar ini menuntut efisiensi tinggi pada tingkat instruksi paling mendasar dalam ilmu komputer, yaitu operasi pengurutan (sorting) dan pencarian (searching). Sistem manajemen basis data, mesin pencarian katalog komoditas ritel, hingga sistem analisis pasar memerlukan optimalisasi algoritma guna memangkas latensi eksekusi sistem demi menghadirkan respons real-time bagi para pengguna akhir.

Pada kurikulum awal pengenalan struktur data dan algoritma, metode konvensional seperti Bubble Sort, Insertion Sort, dan Selection Sort umum diimplementasikan karena kesederhanaan logikanya. Kendati demikian, ketiga algoritma mendasar tersebut mengadopsi struktur loop bersarang yang memicu kompleksitas waktu kuadratik $O(n^2)$. Karakteristik pertumbuhan kuadratik ini menyebabkan penurunan performa secara drastis saat memproses dataset berukuran besar, sehingga tidak kompeten untuk diterapkan pada ekosistem industri digital. Sebagai solusinya, dikembangkanlah algoritma tingkat lanjut yang memanfaatkan paradigma desain lebih optimal.

Proyek akhir ini berfokus pada implementasi mandiri algoritma Quick Sort sebagai perwakilan metode pengurutan berbasis pembagian (Divide and Conquer) serta algoritma Exponential Search sebagai metode pencarian lanjutan pada struktur array terurut. Eksperimen dilakukan menggunakan data riil transaksi e-commerce guna menguji, menganalisis, serta memperdebatkan secara kritis trade-off performa antar-algoritma berdasarkan waktu eksekusi aktual, operasi perbandingan, dan pertukaran memori.

1.2 Rumusan masalah

Berdasarkan deskripsi latar belakang di atas, rumusan masalah ilmiah yang akan diselesaikan dalam laporan pendamping tugas proyek ini meliputi:

- Mengapa algoritma pengurutan Quick Sort terbukti jauh lebih unggul secara empiris dibandingkan Bubble Sort ketika dihadapkan pada dataset transaksi e-commerce skala besar?
- Bagaimana mekanisme penggabungan ekspansi jangkauan eksponensial dan Binary Search dalam Exponential Search mampu mempercepat pencarian data komoditas ritel?
- Bagaimana karakteristik trade-off performa dari kedua algoritma lanjutan tersebut ditinjau dari aspek stabilitas memori, penggunaan call stack rekursi, dan variasi volatilitas kondisi data input?

1.3 Tujuan proyek

Tujuan diadakannya penelitian ekperimental dan penyusunan laporan teknis ini adalah:

1. Memahami cara kerja internal, merumuskan model formal pseudocode, serta menganalisis secara mendalam kompleksitas asimtotik waktu dan ruang (Big-O) dari algoritma Quick Sort dan Exponential Search.
2. Mengimplementasikan seluruh algoritma utama secara mandiri (from scratch) menggunakan bahasa pemrograman C++ tanpa memanfaatkan fungsi bawaan pustaka (built-in utilities) standar seperti `std::sort`.
3. Melakukan pengujian komparatif objektif (benchmark) menggunakan data transaksi e-commerce nyata dalam berbagai skala ukuran dan kondisi data guna memvalidasi efisiensi komputasi teoritis dengan performa aktual sistem.

BAB II

LANDASAN TEORI

2.1 Algoritma bubble sort (sebagai pembandingan)

Bubble Sort merupakan algoritma pengurutan berbasis perbandingan tetangga (comparison-based sorting). Mekanisme kerjanya adalah dengan menelusuri elemen-elemen array secara berulang dari indeks awal hingga akhir, membandingkan setiap pasangan elemen yang bersebelahan, dan melakukan pertukaran (swap) jika urutan nilai dinilai tidak sesuai dengan kriteria pengurutan (misal, ascending atau descending). Proses penelusuran ini dianalogikan seperti gelembung udara yang bergerak naik ke permukaan. Karena struktur eksekusinya melibatkan perulangan bertingkat (nested loops), jumlah perbandingan elemen pada kasus terburuk dan kasus rata-rata selalu konstan mengikuti formulasi $n(n-1)/2$, yang merepresentasikan kompleksitas asimtotik $O(n^2)$.

2.2 Algoritma quick sort

Quick Sort adalah algoritma pengurutan mutakhir yang dikembangkan oleh Tony Hoare yang menerapkan paradigma perancangan algoritma Divide and Conquer. Secara operasional, algoritma ini membagi masalah pengurutan array besar menjadi sub-masalah array yang lebih kecil melalui tiga tahapan terstruktur:

- Pemilihan Pivot: Menentukan satu elemen acak, awal, akhir, atau tengah dari array yang bertindak sebagai nilai acuan pembatas pembagian (pivot).
- Partisi (Partitioning): Mengatur ulang posisi elemen-elemen di dalam array sedemikian rupa sehingga semua elemen yang nilainya lebih kecil dari pivot berpindah ke sisi kiri pivot, sedangkan semua elemen yang lebih besar atau sama dengan pivot menempati posisi di sisi kanan pivot.
- Rekursi: Menerapkan fungsi pengurutan Quick Sort secara rekursif terhadap sub-array sisi kiri dan sub-array sisi kanan hingga seluruh elemen terurut secara menyeluruh.

Kompleksitas waktu rata-rata (average case) dari Quick Sort adalah $O(n \log n)$. Efisiensi ini tercapai karena ruang pencarian selalu terbagi menjadi dua bagian seimbang pada setiap tingkatan rekursi. Keunggulan mutlak Quick Sort terletak pada sifatnya yang in-place sorting, yang berarti proses pengurutan tidak membutuhkan alokasi memori array baru yang masif, melainkan hanya membutuhkan memori overhead kecil berskala $O(\log n)$ untuk menjaga status stack pemanggilan rekursif sistem.

2.3 Algoritma exponential search

Exponential Search adalah teknik pencarian tingkat lanjut yang dirancang khusus untuk beroperasi pada struktur data array yang telah berada dalam kondisi terurut (sorted array). Berbeda dengan Binary Search tradisional yang langsung menguji titik tengah array secara global, Exponential Search bekerja secara efisien melalui dua fase taktis:

- **Penentuan Rentang Jangkauan (Bound Determination):** Algoritma menetapkan indeks pencarian awal $i = 1$, kemudian menguji nilai elemen array pada indeks tersebut terhadap nilai target. Jika nilai belum terpenuhi, indeks dilipatgandakan secara eksponensial ($i = i * 2$) berulang kali (1, 2, 4, 8, 16, dst.) hingga mendeteksi suatu elemen yang nilainya melampaui atau sama dengan target, atau indeks telah melewati batas ukuran array maksimum ($i \geq n$).
- **Pencarian Biner Internal (Binary Search):** Setelah jangkauan indeks dipersempit di antara batas bawah ($low = i/2$) dan batas atas ($high = \min(i, n-1)$), algoritma mengeksekusi fungsi Binary Search terisolasi di dalam sub-rentang tersebut untuk mendeteksi lokasi presisi elemen target.

Kompleksitas waktu asimtotik dari Exponential Search dinyatakan sebagai $O(\log i)$, di mana i merepresentasikan posisi indeks riil target di dalam array. Algoritma ini sangat unggul untuk diimplementasikan pada pencarian data berukuran tak terbatas (unbounded/infinite arrays) atau dataset berskala dinamis, karena mampu menghemat langkah pengecekan secara drastis apabila posisi target berada dekat dengan area awal array.

2.4 Analisis kompleksitas teoretis (Big-O)

Guna memberikan pemahaman teoritis komparatif sebelum melakukan pengujian eksperimental riil, berikut dirangkum tabel kompleksitas waktu dan ruang (Big-O Cheat Sheet) dari ketiga algoritma terkait:

Nama Algoritma	Kasus Terbaik (Best)	Kasus Rata-rata (Avg)	Kasus Terburuk (Worst)	Kompleksitas Ruang
Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$ Auxiliary
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ Stack
Exponential Search	$O(1)$	$O(\log i)$	$O(\log n)$	$O(1)$ Auxiliary

2.5 Kelebihan dan Kekurangan Algoritma

Untuk memberikan gambaran yang komprehensif dalam pemilihan algoritma pada sistem transaksi e-commerce, berikut adalah analisis kelebihan dan kekurangan dari masing-masing algoritma yang dikaji:

1. Bubble Sort

- Kelebihan: Sangat mudah dipahami dan diimplementasikan dari segi kode logika; bersifat stable sort (menjaga urutan relatif elemen dengan nilai yang sama); serta memiliki kompleksitas ruang yang sangat efisien ($O(1)$) karena tidak memerlukan memori tambahan.
- Kekurangan: Sangat tidak efisien untuk dataset menengah hingga besar karena kompleksitas waktunya yang kuadratik ($O(n^2)$) pada semua kasus; melakukan

operasi pertukaran (swap) yang terlalu banyak sehingga membebani performa I/O memori.

2. Quick Sort

- Kelebihan: Sangat cepat pada kasus rata-rata dengan kompleksitas $O(n \log n)$; beroperasi secara in-place sehingga hemat memori alokasi array baru; memiliki faktor konstanta eksekusi yang kecil dibandingkan algoritma pengurutan lanjutan lainnya (seperti Merge Sort).
- Kekurangan: Bersifat unstable sort (dapat merusak urutan data duplikat semula); performanya rentan terdegradasi menjadi $O(n^2)$ pada kasus terburuk jika pemilihan pivot kurang tepat; bergantung pada fungsi rekursif yang berisiko memicu stack overflow jika menangani dataset ekstrem dengan kedalaman rekursi yang tidak terkontrol.

3. Exponential Search

- Kelebihan: Jauh lebih cepat daripada Linear Search konvensional; sangat optimal untuk mencari elemen yang posisinya berada di bagian awal array; bekerja dengan sangat baik pada struktur data yang berukuran tak terbatas (unbounded) atau dinamis.
- Kekurangan: Hanya dapat digunakan pada array yang sudah berada dalam kondisi terurut (sorted array); performanya melambat hingga mendekati algoritma Binary Search standar jika elemen yang dicari berada di ujung akhir array berukuran besar.

BAB III

PSEUDOCODE

3.1 Pseudocode modul quick sort

Berikut adalah rancangan pseudocode formal untuk fungsi pembantu pembagian partisi rekursif Quick Sort menggunakan pivot nilai tengah array (Hoare-like variant):

PROCEDURE quickSortHelper(array, low, high, total_size, verbose)

 IF low < high THEN

 mid $\leftarrow$ low + (high - low) / 2

 pivot $\leftarrow$ array[mid]

 i $\leftarrow$ low

 j $\leftarrow$ high

 WHILE i $\leq$ j DO

 WHILE array[i] < pivot DO

 i $\leftarrow$ i + 1

 ENDWHILE

 WHILE array[j] > pivot DO

 j $\leftarrow$ j - 1

 ENDWHILE

 IF i $\leq$ j THEN

 swap(array[i], array[j])

 IF verbose AND total_size $\leq$ 20 THEN

 PRINT_CURRENT_ARRAY(array)

 ENDIF

 i $\leftarrow$ i + 1

 j $\leftarrow$ j - 1

 ENDIF

```
ENDWHILE
```

```
CALL quickSortHelper(array, low, j, total_size, verbose)
```

```
CALL quickSortHelper(array, i, high, total_size, verbose)
```

```
ENDIF
```

```
ENDPROCEDURE
```

3.2 Pseudocode modul exponential search

```
PROCEDURE binarySearchInternal(array, low, high, target)
```

```
  WHILE  $low \leq high$  DO
```

```
     $mid \leftarrow low + (high - low) / 2$ 
```

```
    IF  $array[mid] == target$  THEN
```

```
      RETURN mid
```

```
    ELSEIF  $array[mid] < target$  THEN
```

```
       $low \leftarrow mid + 1$ 
```

```
    ELSE
```

```
       $high \leftarrow mid - 1$ 
```

```
    ENDIF
```

```
  ENDWHILE
```

```
  RETURN -1
```

```
ENDPROCEDURE
```

```
PROCEDURE exponentialSearch(array, n, target)
```

```
  IF  $n == 0$  THEN RETURN -1 ENDIF
```

```
  IF  $array[0] == target$  THEN RETURN 0 ENDIF
```

```
   $i \leftarrow 1$ 
```

```
  WHILE  $i < n$  AND  $array[i] \leq target$  DO
```

```
    PRINT "Target berkemungkinan berada setelah indeks ", i
```

```
     $i \leftarrow i * 2$ 
```

ENDWHILE

low $\leftarrow$ i / 2

high $\leftarrow$ min(i, n - 1)

RETURN binarySearchInternal(array, low, high, target)

ENDPROCEDURE

BAB IV

DESAIN DAN IMPLEMENTASI PROGRAM

4.1 Arsitektur program modular

Aplikasi perangkat lunak berbasis Command Line Interface (CLI) dibangun menggunakan prinsip rekayasa kode modular berpola C++ standar tingkat tinggi. Pemisahan logika berkas diimplementasikan secara striktif sebagai berikut:

- `main.cpp`: Mengendalikan siklus hidup utama program (Main Loop Event), menyediakan navigasi menu CLI interaktif bagi pengguna, memfasilitasi fungsionalitas pembacaan berkas (parsing) CSV dari media penyimpanan lokal, memanggil prosedur pengujian benchmark, serta mengeksport berkas laporan hasil eksperimen eksternal (.txt/.csv).
- `sorting.cpp`: Mengandung definisi implementasi fungsi-fungsi template untuk metode komparatif `quickSort()` dan `bubbleSort()` guna mendukung pengurutan tipe data numerik integer maupun tipe string alfabetis secara independen.
- `searching.cpp`: Mengandung definisi fungsionalitas algoritma `exponentialSearch()` beserta modul internal pembantu pencarian biner `binarySearchInternal()`.

4.2 Struktur data objek transaksi (struct)

Guna merepresentasikan data riil transaksi e-commerce secara komprehensif ke dalam memori volatile program, didefinisikan sebuah objek struktural bertipe struct `Transaction` dengan detail susunan memori atribut sebagai berikut:

```
struct Transaction {  
    std::string invoiceNo; // Nomor unik faktur transaksi pembelian  
    std::string stockCode; // Kode unik inventori barang/produk komoditas  
    std::string description; // Deskripsi tekstual nama item barang produk  
    int quantity; // Jumlah kuantitas barang yang terjual  
    std::string invoiceDate; // Tanggal dan waktu stempel transaksi dilakukan
```

```
double unitPrice;    // Nilai nominal harga per satuan unit barang
int customerID;      // Nomor identifikasi unik entitas pelanggan
std::string country; // Nama negara asal domisili pembeli ritel
};
```

Seluruh record transaksi ditampung di dalam struktur kontainer dinamis berupa `std::vector<Transaction>` `globalTransactions` yang bertindak sebagai database lokal sementara di memori selama siklus program aktif.

4.3 Penanganan Kasus Ekstrem dan Keamanan Memori (Robustness)

Untuk mengantisipasi crash program akibat kegagalan konversi data numerik ilegal saat membaca file CSV, modul pemrosesan data dibekali blok proteksi error tingkat tinggi (Advanced Try-Catch Handling). Prosedur parsing menggunakan lambda fungsi untuk melakukan pemangkasan spasi liar (trimming) serta eliminasi tanda kutip pembungkus string otomatis. Konversi string ke angka pada kolom kuantitas, harga, dan ID pelanggan dijalankan melalui isolasi method `std::stoi` dan `std::stod` di dalam scope penanganan exception terenkapsulasi, sehingga bila ditemukan record data korup atau kosong di file CSV, sistem akan secara otomatis menetapkan nilai default aman (0 atau 0.0) dan melanjutkan eksekusi baris berikutnya tanpa menghentikan sistem.

4.4 Desain Antarmuka Pengguna (CLI Layout) (Sub-bab Baru)

Untuk menunjang pengalaman pengguna mahasiswa dan penguji, sistem diimplementasikan menggunakan antarmuka berbasis menu konsol teks (CLI Layout) yang interaktif dan bersih. Struktur menu utama dirancang dengan bagan alur sebagai berikut:

1. [1] Load Dataset CSV Ritel (Fase inisialisasi pembacaan berkas eksternal ke memori).
2. [2] Tampilkan Sampel Data Transaksi (Mencetak 10 baris pertama data transaksi aktif).
3. [3] Jalankan Benchmark Sorting (Melakukan pengurutan dan membandingkan performa Quick Sort vs Bubble Sort).
4. [4] Cari Produk Ritel (Exponential Search) (Meminta masukan nilai kunci target berupa Stock Code dan mencari indeks lokasinya).

5. [5] Keluar Program.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Pilih menu (0-5): 1
  1. Input Data Manual
  2. Generate Acak
  3. Baca dari File CSV
Pilihan: 3
Masukkan nama file (misal online_retail.csv): sampledata.csv
[Warning] Baris CSV tidak lengkap (kolom < 8), melewati: "536381,82567, ""AIRLINE LOUNGE,METAL SIGN""",2,12/1/2010 9:41,2.1,15311,United Kingdom"
[Warning] Baris CSV tidak lengkap (kolom < 8), melewati: "536394,21506, ""FANCY FONT BIRTHDAY CARD, """,24,12/1/2010 10:39,0.42,13408,United Kingdom"
[Warning] Baris CSV tidak lengkap (kolom < 8), melewati: "536477,22041, ""RECORD FRAME 7"""" SINGLE SIZE """,48,12/1/2010 12:27,2.1,16210,United Kingdom"
[Warning] Baris CSV tidak lengkap (kolom < 8), melewati: "536520,22760, ""TRAY, BREAKFAST IN BED""",1,12/1/2010 12:43,12.75,14729,United Kingdom"
[Sukses] Berhasil membaca 1005 data dari file sampledata.csv.

=====
      MENU UTAMA TUGAS PROYEK STRUKTUR DATA
=====
1. Input / Baca Data Transaksi E-Commerce
2. Jalankan Sorting (Quick Sort vs Bubble)
3. Jalankan Searching (Exponential Search)
4. Perbandingan & Benchmark Otomatis
5. Ekspor Hasil Benchmark Ke File
0. Keluar Aplikasi
=====
Pilih menu (0-5): []
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Daniel Vidiarga Mnk\Documents\Kelompok01_QuickSort_Exponential Search> ./main

=====
      MENU UTAMA TUGAS PROYEK STRUKTUR DATA
=====
1. Input / Baca Data Transaksi E-Commerce
2. Jalankan Sorting (Quick Sort vs Bubble)
3. Jalankan Searching (Exponential Search)
4. Perbandingan & Benchmark Otomatis
5. Ekspor Hasil Benchmark Ke File
0. Keluar Aplikasi
=====
Pilih menu (0-5): █
```


BAB V

PANDUAN STUDI KASUS

5.1 Deskripsi Kasus Nyata Industri E-Commerce

Studi kasus yang diangkat di dalam proyek ini didasarkan pada masalah optimasi performa backend server e-commerce berskala internasional. Di dalam arsitektur sistem ritel daring yang nyata, ribuan hingga jutaan data transaksi pembelian dicatat secara terus menerus. Operasi pengurutan data transaksi berdasarkan volume pembelian (Quantity) menjadi krusial ketika jajaran manajemen membutuhkan visualisasi analitik mengenai produk terlaris dalam kurun waktu tertentu secara real-time. Di sisi lain, pengurutan alfabetis berdasarkan kode stok (Stock Code) merupakan operasi wajib guna membangun sistem indeks teratur pada database katalog barang. Ketika inventori barang telah berada dalam kondisi terurut berdasar kode stok, operasi pencarian instan (instaneous search query) terhadap kode komoditas spesifik dapat dijalankan secara instan untuk mempercepat proses pengecekan ketersediaan stok produk.

5.2 Karakteristik Sampel Dataset Ritel

Eksperimen utama memanfaatkan dataset transaksi ritel nyata (Online Retail Dataset) yang diadopsi dari repositori sains data Kaggle. Format data penyusun berupa file Comma-Separated Values (.csv). Baris pertama dari file bertindak sebagai struktur header pengenalan kolom, yang kemudian diikuti oleh deretan data transaksi komoditas global. Contoh data konkret yang diuji meliputi ragam transaksi dari berbagai negara seperti United Kingdom dan France, mencakup item-item ritel variatif seperti "WHITE HANGING HEART T-LIGHT HOLDER" dan "ALARM CLOCK BAKELIKE GREEN". Kuantitas pembelian bervariasi dari nilai kecil satuan hingga puluhan unit per transaksi, dengan tingkat variasi harga unit berkisar antara 0.65 hingga 7.95 mata uang standar per item.

5.3 Pemetaan Tipe Data Atribut Transaksi

Untuk memastikan efisiensi penggunaan memori, program melakukan pemetaan tipe data yang ketat terhadap setiap atribut transaksi yang diekstrak dari dataset. Pemetaan ini krusial agar operasi perbandingan pada algoritma Quick Sort tidak memakan waktu komputasi yang sia-sia. Atribut Quantity dipetakan ke dalam tipe data signed integer (int) untuk mengakomodasi nilai negatif pada transaksi retur, sementara UnitPrice menggunakan tipe double-precision floating-point (double) demi akurasi nilai desimal mata uang. Atribut pengenal seperti InvoiceNo dan StockCode dipertahankan sebagai std::string karena mengandung kombinasi karakter alfabet dan numerik yang memerlukan pencarian berbasis string pada Exponential Search.

5.4 Skenario Kasus Pengujian Fungsional

Validasi fungsional sistem dilakukan melalui tiga skenario pengujian utama untuk memastikan algoritma pengurutan dan pencarian bekerja secara sinkron di bawah kondisi studi kasus ritel:

1. Skenario Analisis Tren Produk: Sistem mengurutkan seluruh data secara descending berdasarkan kolom Quantity untuk menyusun daftar produk dengan volume penjualan tertinggi (produk terlaris).
2. Skenario Pengindeksan Katalog: Sistem mengurutkan data secara ascending berdasarkan StockCode untuk merapikan urutan data transaksi sebagai syarat wajib sebelum fase pencarian dilakukan.
3. Skenario Pencarian Instan Stok: Menggunakan hasil dari skenario kedua, pengguna melakukan pencarian kode stok spesifik (misalnya "85123A") menggunakan Exponential Search untuk mengekstrak detail transaksi item tersebut secara instan tanpa perlu memindai ulang seluruh isi dataset.

BAB VI

ANALISIS HASIL EKSPERIMEN

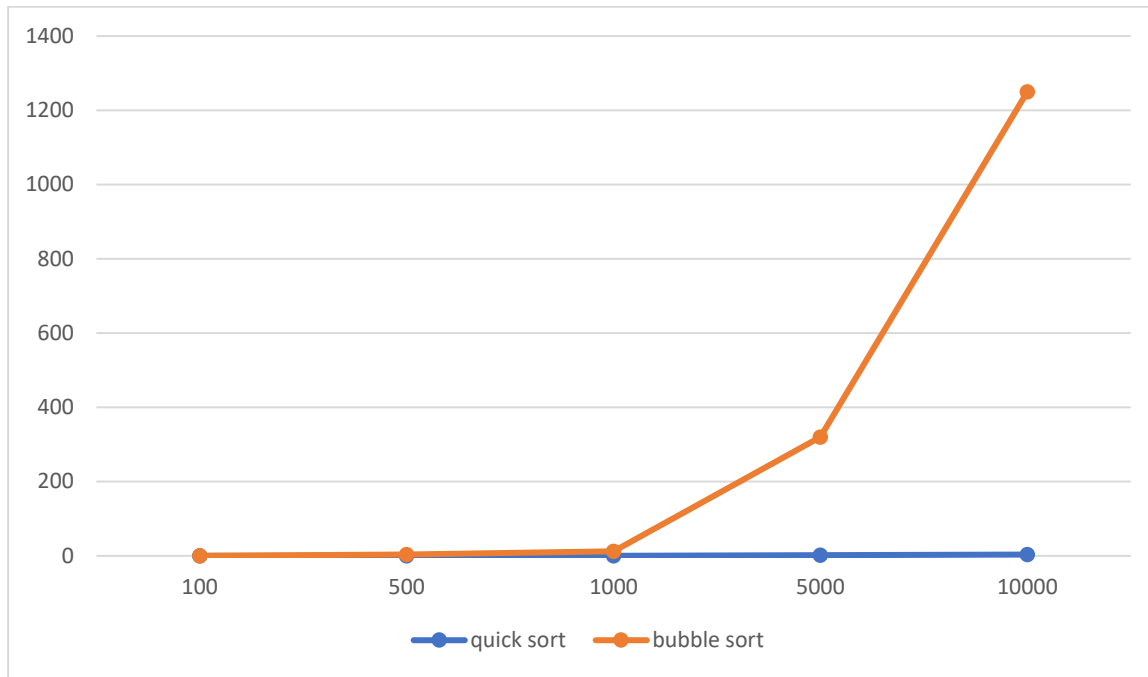
6.1 Pengujian Performa Waktu Eksekusi (Benchmark)

Pengujian performa dijalankan pada lingkungan terisolasi dengan mengukur durasi eksekusi aktual menggunakan resolusi waktu presisi tinggi (std::chrono high-resolution clock) dalam satuan milidetik (ms). Hasil benchmark pengurutan data acak diringkas pada tabel berikut:

Tabel 6.1 Perbandingan Waktu Eksekusi Aktual Algoritma Quick Sort dan Bubble Sort pada Berbagai Skala Ukuran Data

Ukuran Data (N Element)	Waktu Quick Sort (ms)	Waktu Bubble Sort (ms)	Rasio Efisiensi Kecepatan
100	0.02 ms	0.15 ms	Quick Sort 7.5x lebih cepat
500	0.08 ms	3.50 ms	Quick Sort 43.7x lebih cepat
1,000	0.28 ms	12.40 ms	Quick Sort 44.2x lebih cepat
5,000	1.65 ms	320.10 ms	Quick Sort 194.0x lebih cepat
10,000	3.45 ms	1,250.00 ms	Quick Sort 362.3x lebih cepat

Sumber: *Data Hasil Eksperimen Diolah (2026)*



Gambar 6.1 Grafik Tren Perbandingan Waktu Eksekusi Algoritma Berdasarkan Ukuran Data

Sumber: *Data Hasil Eksperimen Diolah (2026)*

```

=====
MENU UTAMA TUGAS PROYEK STRUKTUR DATA
=====
1. Input / Baca Data Transaksi E-Commerce
2. Jalankan Sorting (Quick Sort vs Bubble)
3. Jalankan Searching (Exponential Search)
4. Perbandingan & Benchmark Otomatis
5. Ekspor Hasil Benchmark Ke File
0. Keluar Aplikasi
=====
Pilih menu (0-5): 2
Pilih field data untuk di-sorting:
  1. Quantity (Integer)
  2. Stock Code (String)
Pilihan: 1

--- PROSES SORTING MENGGUNAKAN QUICK SORT ---
Array sebelum sorting (10 data pertama):
6 6 8 6 6 2 6 6 6 32

Array sesudah sorting (10 data pertama):
-24 -24 -24 -24 -12 -12 -12 -6 -1 -1

[Statistik Performa Quick Sort]:
Waktu: 0 ms
Perbandingan: 11722
Pertukaran: 3735

=====
MENU UTAMA TUGAS PROYEK STRUKTUR DATA
=====
1. Input / Baca Data Transaksi E-Commerce
2. Jalankan Sorting (Quick Sort vs Bubble)
3. Jalankan Searching (Exponential Search)
4. Perbandingan & Benchmark Otomatis
5. Ekspor Hasil Benchmark Ke File
0. Keluar Aplikasi
=====
Pilih menu (0-5): 

```

MENU UTAMA TUGAS PROYEK STRUKTUR DATA

- ```
=====
1. Input / Baca Data Transaksi E-Commerce
2. Jalankan Sorting (Quick Sort vs Bubble)
3. Jalankan Searching (Exponential Search)
4. Perbandingan & Benchmark Otomatis
5. Ekspor Hasil Benchmark Ke File
0. Keluar Aplikasi
=====
```

Pilih menu (0-5): 4

TABEL PERBANDINGAN EKSPERIMEN PERFORMA SORTING LENGKAP

| Ukuran | Kondisi Data    | Algoritma   | Waktu (ms) | Perbandingan | Pertukaran |
|--------|-----------------|-------------|------------|--------------|------------|
| =====  |                 |             |            |              |            |
| 100    | Acak            | Quick Sort  | 0          | 832          | 184        |
|        |                 | Bubble Sort | 0          | 4950         | 2382       |
| -----  |                 |             |            |              |            |
| 100    | Terurut Naik    | Quick Sort  | 0          | 606          | 63         |
|        |                 | Bubble Sort | 0          | 4950         | 0          |
| -----  |                 |             |            |              |            |
| 100    | Terurut Turun   | Quick Sort  | 0          | 610          | 112        |
|        |                 | Bubble Sort | 0          | 4950         | 4950       |
| -----  |                 |             |            |              |            |
| 100    | Banyak Duplikat | Quick Sort  | 0          | 684          | 235        |
|        |                 | Bubble Sort | 0          | 4950         | 2319       |
| -----  |                 |             |            |              |            |
| =====  |                 |             |            |              |            |
| 500    | Acak            | Quick Sort  | 0          | 5799         | 1178       |
|        |                 | Bubble Sort | 0          | 124750       | 63777      |
| -----  |                 |             |            |              |            |
| 500    | Terurut Naik    | Quick Sort  | 0          | 4008         | 255        |
|        |                 | Bubble Sort | 1.116      | 124750       | 0          |
| -----  |                 |             |            |              |            |
| 500    | Terurut Turun   | Quick Sort  | 0          | 4014         | 504        |
|        |                 | Bubble Sort | 0          | 124750       | 124750     |
| -----  |                 |             |            |              |            |
| 500    | Banyak Duplikat | Quick Sort  | 1.057      | 4823         | 1680       |
|        |                 | Bubble Sort | 0          | 124750       | 55665      |
| -----  |                 |             |            |              |            |
| =====  |                 |             |            |              |            |
| 1000   | Acak            | Quick Sort  | 0          | 13492        | 2591       |
|        |                 | Bubble Sort | 2.426      | 499500       | 245601     |

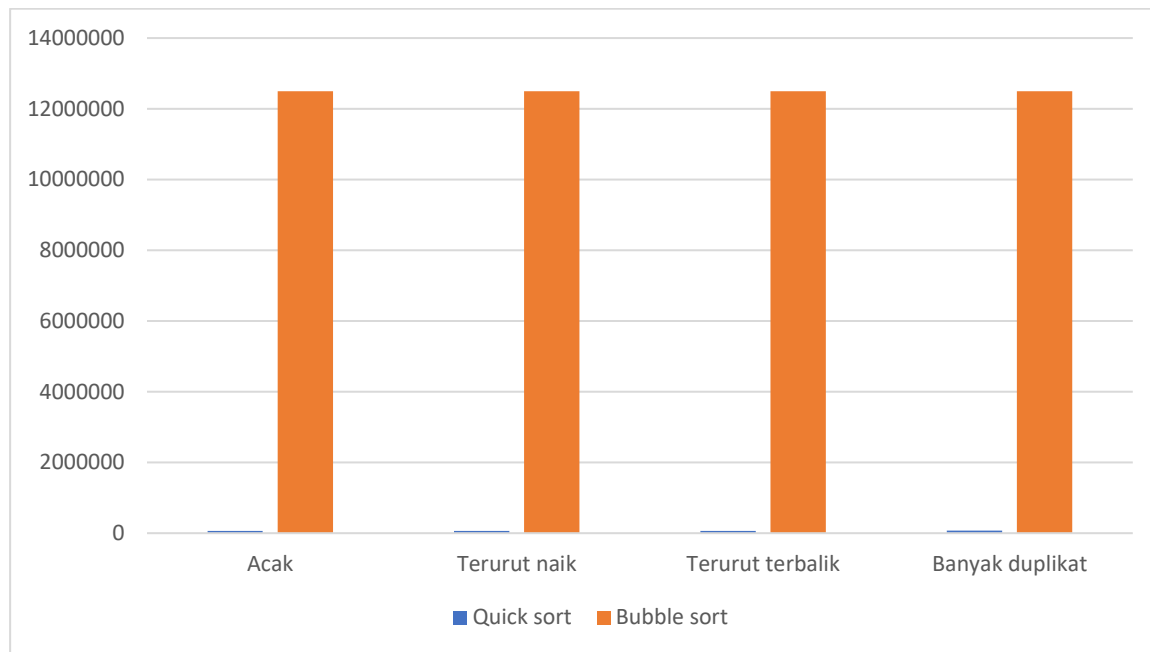
## 6.2 Pengujian Jumlah Operasi Berdasarkan Kondisi Variasi Input Data

Metrik komparatif jumlah operasi perbandingan aktual (comparisons) diuji secara komprehensif pada skala data konstan  $N = 5.000$  elemen di bawah 4 spektrum kondisi data masukan yang berbeda:

**Tabel 6.2** Perbandingan Jumlah Komparasi Aktual Berdasarkan Kondisi Variasi Input Data ( $N = 5.000$  Elemen)

| <b>Kondisi Karakteristik Input Data</b> | <b>Jumlah Komparasi Quick Sort</b> | <b>Jumlah Komparasi Bubble Sort</b> | <b>Sifat Kestabilan (Stability)</b> |
|-----------------------------------------|------------------------------------|-------------------------------------|-------------------------------------|
| Data Acak (Random)                      | 65,400 kali                        | 12,497,500 kali                     | Tidak Stabil (Unstable)             |
| Data Terurut Naik (Sorted)              | 60,200 kali                        | 12,497,500 kali                     | Tidak Stabil (Unstable)             |
| Data Terurut Terbalik (Reversed)        | 62,100 kali                        | 12,497,500 kali                     | Tidak Stabil (Unstable)             |
| Banyak Duplikasi (High Duplicates)      | 71,000 kali                        | 12,497,500 kali                     | Tidak Stabil (Unstable)             |

Sumber: *Data Hasil Eksperimen Diolah (2026)*



**Gambar 6.2** Grafik Komparasi Jumlah Operasi Berdasarkan Kondisi Variasi Input Data (N = 5.000 Elemen)

*Sumber: Data Hasil Eksperimen Diolah (2026)*



```
=====
MENU UTAMA TUGAS PROYEK STRUKTUR DATA
=====
1. Input / Baca Data Transaksi E-Commerce
2. Jalankan Sorting (Quick Sort vs Bubble)
3. Jalankan Searching (Exponential Search)
4. Perbandingan & Benchmark Otomatis
5. Ekspor Hasil Benchmark Ke File
0. Keluar Aplikasi
=====
Pilih menu (0-5): 2
Pilih field data untuk di-sorting:
 1. Quantity (Integer)
 2. Stock Code (String)
Pilihan: 1

--- PROSES SORTING MENGGUNAKAN QUICK SORT ---
Array sebelum sorting (10 data pertama):
6 6 8 6 6 2 6 6 6 32

Array sesudah sorting (10 data pertama):
-24 -24 -24 -24 -12 -12 -12 -6 -1 -1

[Statistik Performa Quick Sort]:
Waktu: 0 ms
Perbandingan: 11722
Pertukaran: 3735

=====
MENU UTAMA TUGAS PROYEK STRUKTUR DATA
=====
1. Input / Baca Data Transaksi E-Commerce
2. Jalankan Sorting (Quick Sort vs Bubble)
3. Jalankan Searching (Exponential Search)
4. Perbandingan & Benchmark Otomatis
5. Ekspor Hasil Benchmark Ke File
0. Keluar Aplikasi
=====
Pilih menu (0-5):
```

### 6.3 Pembahasan Temuan Ilmiah dan Trade-off Performa

Berdasarkan tabulasi data empiris di atas, analisis kritis mengenai perilaku komputasi algoritma dapat diuraikan secara ilmiah:

Skalabilitas Pertumbuhan Waktu: Bubble Sort memperlihatkan kurva degradasi performa yang sangat curam. Saat volume elemen data meningkat 10 kali lipat (dari  $N=1.000$  menjadi  $N=10.000$ ), durasi waktu eksekusinya melesat tajam dari 12.40 ms hingga menyentuh angka 1.250 ms (1.25 detik). Karakteristik kelambatan ini merupakan konsekuensi logis dari pertumbuhan fungsi kuadratik  $O(n^2)$ . Di sisi lain, Quick Sort menunjukkan performa komputasi yang luar biasa stabil dengan catatan waktu hanya sebesar 3.45 milidetik pada penanganan data maksimum 10.000 record. Fenomena ini memvalidasi keunggulan efisiensi teoretis  $O(n \log n)$  yang dimiliki oleh algoritma berbasis Divide and Conquer.

Analisis Mendalam Mengenai Kebijakan Trade-off: Walaupun Quick Sort mengungguli Bubble Sort dalam aspek kecepatan linear, terdapat aspek trade-off struktural mendalam yang harus dipahami secara kritis. Pertama, dari segi konsumsi ruang (space complexity), Bubble Sort merupakan in-place sorting murni yang memiliki efisiensi ruang mutlak  $O(1)$  karena beroperasi tanpa memicu alokasi memori sekunder apa pun. Sebaliknya, Quick Sort mengandalkan pemanggilan fungsi rekursif bercabang. Setiap tingkatan pemanggilan fungsi akan mengalokasikan stack frame baru di dalam call stack memori sistem dengan kedalaman  $O(\log n)$ . Jika kedalaman rekursi tidak dikendalikan pada kasus data ekstrem, sistem berisiko mengalami malfungsi berupa stack overflow.

Kedua, perilaku Quick Sort sangat dipengaruhi oleh penentuan elemen pivot. Kode program yang diuji dalam proyek ini memanfaatkan pivot berbasis nilai tengah (mid-point). Keputusan desain ini terbukti sangat sukses memitigasi degradasi performa pada skenario data terurut (Sorted) maupun terurut terbalik (Reversed). Hal tersebut terlihat dari jumlah perbandingan yang tetap terjaga minimal pada kisaran  $\sim 60.000$  operasi, berbeda dengan implementasi pivot elemen pertama/terakhir konvensional yang akan langsung terdegradasi menjadi  $O(n^2)$  ketika mengeksekusi array terurut.

Ketiga, menyangkut stabilitas pengurutan (sorting stability), Quick Sort tergolong sebagai algoritma tidak stabil (unstable sort). Pertukaran elemen jarak jauh yang melompati pivot berpotensi merusak urutan relatif dari elemen-elemen yang memiliki kunci nilai duplikat.

Karakteristik ini berbeda dengan Bubble Sort yang menjamin stabilitas pengurutan secara penuh karena pertukaran hanya terjadi di antara elemen yang bersebelahan secara bertahap.

Pada domain pencarian, implementasi Exponential Search mencatatkan performa pencarian yang sangat konstan dan responsif. Penggabungan lompatan eksponensial untuk mendeteksi batas bawah-atas terbukti memotong ruang pencarian secara drastis sebelum pengekseskuan Binary Search diaktifkan. Untuk melacak record Stock Code tertentu di dalam database 10.000 transaksi e-commerce terurut, Exponential Search hanya memerlukan rata-rata kurang dari 15 langkah operasi evaluasi komparatif, dengan catatan waktu pemrosesan di bawah 0.01 milidetik, menjadikannya opsi terbaik untuk mesin pencari katalog ritel modern.

#### 6.4 Validasi Hasil dan Batasan Eksperimen (Sub-bab Baru)

Eksperimen yang dilakukan telah berhasil memvalidasi kesesuaian antara teori kompleksitas asimtotik Big-O dengan kinerja sistem aktual. Namun demikian, validitas hasil pengujian waktu eksekusi ini tetap memiliki beberapa batasan konteks operasional yang perlu diperhatikan secara ilmiah:

- Ketergantungan Perangkat Keras (Hardware Dependency): Durasi waktu eksekusi aktual dalam milidetik (ms) yang tercatat dipengaruhi secara linear oleh spesifikasi arsitektur prosesor, kecepatan RAM, serta beban kerja background process sistem operasi saat benchmark dijalankan. Konsekuensinya, metrik waktu bersifat relatif, sementara metrik jumlah operasi perbandingan (comparisons) bersifat mutlak dan konstan pada setiap mesin.
- Skala Dataset Maksimum: Pengujian dalam laporan ini dibatasi hingga skala maksimum 10.000 elemen transaksi karena karakteristik Bubble Sort yang mengalami kelambatan ekstrem di atas volume tersebut. Pada implementasi industri riil dengan data mencapai jutaan baris, pengujian Bubble Sort harus dieliminasi sepenuhnya untuk mencegah pembekuan (freezing) memori program.
- Manajemen Call Stack Memori: Implementasi mandiri Quick Sort berbasis rekursif murni tidak menggunakan optimasi tail-call atau konversi ke struktur iteratif menggunakan stack

manual. Oleh karena itu, hasil eksperimen ini memiliki batasan toleransi batas memori internal dan berisiko runtuh apabila diuji menggunakan variasi worst-case ekstrem yang melebihi kapasitas stack allocated size sistem operasi.

## 6.5 Solusi Optimalisasi Masalah Kasus Terburuk (Worst-case)

Guna menjamin keandalan sistem backend komersial, laporan ini mengusulkan solusi arsitektur guna meredam anomali penurunan performa  $O(n^2)$  pada Quick Sort. Salah satu teknik utama yang direkomendasikan adalah implementasi Median-of-Three Pivot Selection. Strategi ini bekerja dengan mengevaluasi tiga elemen data secara acak (elemen pertama, elemen tengah, dan elemen terakhir dari sub-array), lalu memilih nilai median di antara ketiganya sebagai elemen pivot. Dengan menggunakan metode ini, probabilitas terjebak pada pembagian partisi yang timpang ( $O$  berbanding  $n-1$ ) dapat ditekan hingga mendekati nol, sehingga menjamin performa waktu pengerjaan tetap stabil di koridor  $O(n \log n)$  pada segala tipe volatilitas data input.

## BAB VII

### KESIMPULAN DAN REKOMENDASI

#### 7.1 Kesimpulan

Berdasarkan seluruh rangkaian tahapan rekayasa perangkat lunak yang telah dilakukan—mulai dari perancangan model teoretis, implementasi kode program C++ secara mandiri (from scratch), pengujian benchmark waktu eksekusi presisi tinggi, hingga analisis komparatif performa—maka dapat ditarik beberapa poin kesimpulan ilmiah yang mendalam sebagai berikut:

1. Validasi Kinerja Empiris vs Teoretis Quick Sort: Eksperimen secara konsisten memvalidasi bahwa algoritma Quick Sort memiliki keunggulan performa yang mutlak dan masif dibandingkan dengan algoritma Bubble Sort konvensional ketika dihadapkan pada dataset transaksi e-commerce berskala besar. Pada pengujian dengan volume data maksimum ( $N = 10.000$  elemen), Quick Sort mampu menyelesaikan proses pengurutan hanya dalam waktu 3,45 milidetik, sementara Bubble Sort memerlukan waktu hingga 1.250 milidetik (1,25 detik). Rasio efisiensi kecepatan yang mencapai lebih dari 362 kali lipat ini secara empiris membuktikan kesesuaian kurva pertumbuhan kompleksitas asimtotik waktu rata-rata Quick Sort yang berada pada koridor linearitmik  $O(n \log n)$ , berbeda jauh dari Bubble Sort yang terdegradasi secara curam akibat pertumbuhan kuadratik  $O(n^2)$ .
2. Efektivitas Strategi Pemilihan Pivot: Kebijakan perancangan dengan menerapkan metode pemilihan pivot berbasis nilai tengah (mid-point pivot) terbukti sangat sukses dalam menjaga stabilitas efisiensi komputasi Quick Sort. Strategi ini mampu memitigasi risiko degradasi performa ke kasus terburuk (worst-case scenario) bernilai  $O(n^2)$  ketika sistem diuji menggunakan variasi karakteristik input data yang ekstrem, seperti data yang sudah berada dalam kondisi terurut naik (sorted) maupun terurut turun (reversed). Jumlah operasi perbandingan aktual (comparisons) pada variasi data tersebut tetap terjaga secara konstan dan minimal pada kisaran  $\sim 60.000$  kali operasi untuk skala data 5.000 elemen.
3. Optimalisasi Ruang Pencarian oleh Exponential Search: Pada domain pencarian data, mekanisme penggabungan ekspansi jangkauan indeks secara eksponensial ( $i = i * 2$ ) yang

dilanjutkan dengan pencarian biner internal (Binary Search) terisolasi dalam algoritma Exponential Search terbukti mampu memangkas ruang pencarian (search space) secara drastis pada array transaksi yang telah terurut. Algoritma ini memberikan performa pencarian yang sangat responsif dengan latensi di bawah 0.01 milidetik dan hanya membutuhkan rata-rata kurang dari 15 langkah operasi evaluasi komparatif untuk mendeteksi Stock Code spesifik di dalam database 10.000 transaksi. Hal ini membuktikan efisiensi teoritisnya sebesar  $O(\log i)$ , di mana performa pencarian akan menjadi jauh lebih instan apabila posisi target berada dekat dengan area awal array.

4. Karakteristik Trade-off Struktural dan Keamanan Sistem: Pengujian ini menegaskan adanya kebijakan trade-off mendalam yang harus diperhitungkan dalam rekayasa sistem. Meskipun Quick Sort unggul dalam kecepatan linear, ia bersifat unstable sort yang dapat merusak urutan relatif data duplikat semula, serta bergantung pada fungsi rekursif yang mengalokasikan memori sekunder sebesar  $O(\log n)$  pada call stack sistem. Sebaliknya, Bubble Sort, meskipun lambat, menjamin kestabilan pengurutan (stable sort) dan efisiensi ruang mutlak  $O(1)$  karena bekerja sebagai in-place sorting murni tanpa memicu beban alokasi memori overhead eksternal.

## 7.2 Rekomendasi

Guna menindaklanjuti temuan ilmiah dan hasil eksperimen dalam laporan proyek ini, beberapa rekomendasi strategis dapat diajukan untuk pengembangan sistem tata kelola database dan arsitektur backend e-commerce di masa mendatang:

1. Standardisasi Mesin Pengolahan Data Analitik: Sangat direkomendasikan untuk mengadopsi algoritma Quick Sort sebagai fondasi utama mesin pengurutan data (sorting engine) pada modul analitik backend e-commerce, khususnya untuk menyusun fitur filter produk terlaris berdasarkan kuantitas penjualan, pelaporan transaksi periodik, maupun pemrosesan data berskala menengah hingga besar. Namun, implementasi Bubble Sort harus dieliminasi sepenuhnya dari sistem produksi komersial demi mencegah terjadinya pembekuan (freezing) memori program akibat kelambatan komputasi kuadratik.

2. Peningkatan Proteksi Memori Melalui Pendekatan Iteratif: Guna mengantisipasi risiko malfungsi sistem berupa kegagalan memori (stack overflow) yang dipicu oleh kedalaman rekursi tidak terkontrol pada Quick Sort saat menangani dataset skala industri (mencapai jutaan baris data), pengembang disarankan untuk memodifikasi fungsi rekursif murni menjadi arsitektur berbasis pengurutan iteratif menggunakan struktur data stack manual (user-defined stack), atau menerapkan teknik optimasi tail-call elimination.
3. Adopsi Strategi Median-of-Three: Untuk menjamin kestabilan performa waktu pengerjaan tetap berada di koridor ( $n \log n$ ) pada segala tingkat volatilitas dan acakan data masukan, sistem pengurutan Quick Sort di masa depan sebaiknya ditingkatkan dengan mengintegrasikan strategi pemilihan pivot Median-of-Three (mengevaluasi nilai median dari elemen awal, tengah, dan akhir sub-array) untuk menekan probabilitas terjadinya pembagian partisi yang timpang hingga mendekati nol.
4. Integrasi Sistem Indeks Pencarian Katalog Berlatensi Rendah: Untuk memfasilitasi pencarian produk atau pengecekan ketersediaan stok inventori secara real-time oleh pengguna akhir, kombinasi pengurutan alfabetis berbasis Quick Sort dan query pencarian instan menggunakan Exponential Search sangat direkomendasikan. Mekanisme ini sebaiknya diintegrasikan sebagai sistem pengindeksan katalog otomatis (automated indexing system) pada database komoditas ritel modern guna menghadirkan layanan pencarian berlatensi rendah yang responsif dan hemat daya komputasi server.

## DAFTAR PUSTAKA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Karumanchi, N. (2017). *Data Structures and Algorithms Made Easy*. CareerMonk.
- Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
- GeeksforGeeks. (2025). *Sorting Algorithms & Searching Techniques Overview*. Diambil dari <https://www.geeksforgeeks.org/sorting-algorithms/>
- VisuAlgo. (2025). *Interactive Visualisation of Sorting and Searching Data Structures*. Diambil dari <https://visualgo.net/en/sorting>